Operación: El Código Perdido

TP3 — Búsqueda Lineal y Binaria en Java

Un archivo secreto con miles de registros ha sido robado de una agencia de investigación. La única copia que queda está desordenada, mezclada a propósito para que nadie pueda encontrar nada rápido. Como analistas de datos su misión es construir el software que la agencia usará para buscar sospechosos, pistas y códigos ocultos dentro de este archivo.

Ustedes deberán implementar, comparar y analizar búsqueda lineal y búsqueda binaria sobre una estructura de datos propia, midiendo su rendimiento y tomando decisiones justificadas.

Objetivos de aprendizaje

- Implementar búsqueda lineal y búsqueda binaria en Java desde cero (sin usar Arrays.binarySearch ni librerías equivalentes).
- Explicar por qué la búsqueda binaria exige una colección ordenada y qué costo tiene ordenar.
- Medir y comparar el rendimiento real de ambos algoritmos con distintos tamaños de datos.
- Elegir el algoritmo correcto según el contexto (datos ordenados vs. desordenados, búsquedas puntuales vs. repetidas).
- Trabajar con arreglos o colecciones (ArrayList), clases propias y buenas prácticas de organización de código.
- No hace falta leer ningún archivo ni generar los datos, eso ya viene resuelto, ustedes recibirán la lista para usar, una clase con un ArrayList<Sospechoso> cargado con 500 registros ().

Cada Sospechoso tiene al menos estos campos: int codigo; String alias; String ciudad; int nivelDeRiesgo;

Consigna

Los alumnos reciben, ya lista para usar, una clase con un ArrayList<Sospechoso> cargado con 500 registros (no hace falta leer ningún archivo ni generar los datos: eso ya viene resuelto). Cada Sospechoso tiene al menos estos campos:

```
public class Sospechoso {

    int codigo;

    String alias;

    String ciudad;

    int nivelDeRiesgo;

}
```

1. Archivo en bruto — Búsqueda lineal

- Usar la lista ya provista (500 sospechosos, desordenados) sin modificarla.
- Implementar buscarPorCodigoLineal(int codigo) usando búsqueda lineal manual.
- Implementar una variante creativa, buscarSospechososPorCiudadYRiesgo(...), que devuelva todos los que cumplen dos condiciones (esto muestra que la lineal no sirve solo para "encontrar uno", sino también para filtrar).
- Medir el tiempo de búsqueda con System.nanoTime() para códigos ubicados al principio, en el medio y al final de la lista y en posiciones intermedio, imprimir y anotar los resultados.

2. Archivo clasificado — Búsqueda binaria

La agencia consigue una versión ordenada del archivo (por código). Ahora hay que demostrar que existe una forma mucho más eficiente de buscar.

- Ordenar la lista por código (puede usarse Collections.sort con Comparable/Comparator, ya que el foco del TP es la búsqueda, no el ordenamiento). El tiempo de ordenación debe tenerse en cuenta en su conclusión.
- Implementar buscarPorCodigoBinaria(int codigo) de forma manual, iterativa o recursiva.
- Repetir las mismas mediciones de tiempo que con la búsqueda lineal, sobre los mismos datos, y comparar. Generar informe con sus conclusiones.

- Generar una pequeña tabla o gráfico, los datos usados para generar la tabla deben estar impreso por consola, mostrando tiempo vs. cantidad de elementos (por ejemplo con 500, 5.000, 50.000 y 500.000 registros) para lineal vs. binaria.
- Implementar una variante creativa, un código no está en la lista, pero el sistema debe responder **"no encontrado, pero el sospechoso más parecido en código es X"**. Esto le obliga a pensar qué hace la búsqueda binaria cuando no encuentra el valor exacto, trabajando con los índices low/high en el momento del corte.

Entregables

- Código fuente en Java, organizado en al menos estas clases: Sospechoso, GeneradorDeDatos (o main con carga de datos), BuscadorLineal, BuscadorBinario, todo en el **"package tp3".**
- Un informe con sus conclusiones sobre la comparación entre la búsqueda lineal y la búsqueda binaria, que incluya lo que se detalla a continuación.

	Búsqueda Lineal	Búsqueda Binaria
Mejor Caso	$O(1)$	$O(1)$
Caso Promedio	$O((n+1)/2)$	$O(\log n)$
Peor Caso	$O(n)$	$O(\log n)$

Observe la imagen que compara la complejidad de los algoritmos y explique con sus propias palabras el comportamiento de cada algoritmo en los siguientes casos:

1. ¿Cómo están conformado los datos en estos casos? Explique cuál es el mejor, promedio y peor caso.
2. **Mejor caso:** explique por qué tanto la búsqueda lineal como la búsqueda binaria pueden tener una complejidad **O(1)**.
3. **Caso promedio**: explique por qué la búsqueda lineal necesita, en promedio, aproximadamente **O((n+1)/2)** comparaciones, mientras que la búsqueda binaria presenta una complejidad **O(log n)**.
4. **Peor caso:** explique por qué la búsqueda lineal puede necesitar recorrer los **n elementos — O(n)**, mientras que la búsqueda binaria mantiene una complejidad **O(log n)**.
5. ¿Qué le pasaría a la búsqueda lineal si el archivo tuviera 10 millones de registros?

En su respuesta, incluya un gráfico con **los tiempos de la ejecución para cada caso** y explique qué característica de la búsqueda binaria permite que sea más eficiente que la búsqueda lineal cuando aumenta la cantidad de elementos.

Además el grafico debe tener los tiempos de por lo menos más 20 ejecuciones, **10 entre el caso 1 y 2**, **10 entre el caso 2 y 3**, variando cantidad de elementos en la lista.

El Informe debe ser un archivo **.doc** breve (1–2 páginas) con: tabla y gráfico comparativo de tiempos; respuesta a las preguntas; y una **conclusión sobre en qué casos reales usarían cada algoritmo**.

- Repositorio o carpeta comprimida con todo el proyecto x, listo para compilar y ejecutar.